

Dokumentacja projektu: apple_microbench_variant2_streamfix

1. Cel projektu

Celem repozytorium jest dostarczenie mikrobenchmarków CPU i GPU dla wielu platform, z możliwie spójnym modelem uruchamiania, zapisu wyników CSV oraz analizą podsumowującą.

Główne klasy benchmarków:

- przepustowość pamięci,
- STREAM (copy/scale/add/triad),
- latencja pointer-chasing,
- wydajność obliczeniowa FMA,
- peak FMA (maksymalne obciążenie jednostek obliczeniowych).

2. Struktura projektu

2.1 CPU

Kluczowe pliki:

- `cpu/lib/microbench.c`, `cpu/lib/microbench.h`
- `cpu/benchmarks/run\_bandwidth.py`
- `cpu/benchmarks/run\_bandwidth\_mt.py`
- `cpu/benchmarks/run\_pointer\_latency.py`
- `cpu/benchmarks/run\_stream.py`
- `cpu/benchmarks/run\_stream\_mt.py`
- `cpu/benchmarks/run\_compute\_fma.py`
- `cpu/benchmarks/run\_compute\_fma\_peak.py`

Dostępne są też warianty katalogowe per-vendor:

- `cpu/intel/benchmarks/\*`
- `cpu/amd/benchmarks/\*`

Biblioteka CPU:

- Linux: `cpu/lib/libmicrobench.so`
- macOS: `cpu/lib/libmicrobench.dylib`
- build skrypty: `cpu/lib/build\_linux.sh`, `cpu/lib/build\_mac.sh`

2.2 GPU

Backendy i benchmarki:

- CUDA: `gpu/cuda/benchmarks/\*`
- HIP: `gpu/hip/benchmarks/\*`
- Metal: `gpu/metal/benchmarks/\*`
- OpenCL: `gpu/opengl/benchmarks/\*`
- Intel (oddzielny katalog): `gpu/intel/benchmarks/\*`

Backendy implementacyjne:

- `gpu/cuda/cuda\_backend.py`
- `gpu/hip/hip\_backend.py`
- `gpu/metal/metal\_backend.py`
- `gpu/opengl/opengl\_backend.py`

2.3 Orkiestracja

- `run\_all\_cpu\_benchmarks.py`
- `run\_all\_gpu\_benchmarks.py`
- `run\_all\_benchmarks.py`
- `run\_all\_backends.py`

2.4 Energia i moc

- `energy.py` (CPU: RAPL / powermetrics)
- `energy\_utils.py` (CPU + GPU logger, NVML/sysfs/powermetrics)

2.5 Analiza

- `analysis/cpu\_summary.py`

- `analysis/gpu\_summary.py`

2.6 Dane

- CPU CSV: `data/cpu/\*.csv`
- GPU CSV: `data/gpu/\*.csv`

3. Aktualny przepływ uruchomienia

3.1 CPU

1. `run\_all\_cpu\_benchmarks.py` sprawdza i ewentualnie buduje bibliotekę C.
2. Uruchamiane są benchmarki CPU po kolei.
3. Po sukcesie uruchamiane jest `analysis/cpu\_summary.py`.

3.2 GPU

1. `run\_all\_gpu\_benchmarks.py` wybiera backend (`auto` lub jawny).
2. Uruchamiane są benchmarki bandwidth/FMA/FMA_peak dla backendu.
3. Wyniki trafiają do `data/gpu`.
4. Opcjonalnie uruchamiane jest `analysis/gpu\_summary.py` (z `run\_all\_benchmarks.py`).

4. Co działa dobrze obecnie

- Jest pełny zestaw benchmarków CPU (memory/latency/compute/stream).
- Jest wiele backendów GPU (CUDA/HIP/Metal/OpenCL).
- Jest orkiestracja end-to-end dla CPU i GPU.
- Jest mechanizm zapisu metadanych do CSV (CPU model, architektura, backendy).
- Jest podstawowe podsumowanie wyników przez skrypty w `analysis/`.

5. Zidentyfikowane luki i ryzyka techniczne

5.1 Nieśpójne nazewnictwo benchmarków CPU

W kilku skryptach wartość pola `benchmark` nie odpowiada temu, czego oczekuje `analysis/cpu\_summary.py`.
Skutek: część danych może nie być uwzględniana w podsumowaniu.

5.2 Nieśpójny schemat kolumn CPU CSV

Różne benchmarki zapisują różne nazwy pól (np. `size\_bytes` vs `size\_mb`, `working\_set\_bytes` vs oczekiwane pola w summary).
Skutek: niepełne agregacje i trudniejsze porównania między benchmarkami.

5.3 Błąd typu danych w pointer-chasing

W `run\_pointer\_latency.py` tablica indeksów jest tworzona jako `np.uint64`, a do kernela przekazywana jako wskaźnik `uint32`.
Skutek: ryzyko błędnych pomiarów latencji.

5.4 Dwa równoległe mechanizmy pomiaru energii

CPU benchmarki korzystają głównie z `energy.py`, a GPU z `energy\_utils.py`; model pomiaru i pola wynikowe nie są identyczne.
Skutek: ograniczona porównywalność metryk energii i mocy.

5.5 Nieśpójność jednostek przepustowości

W części kodu przepustowość liczona jest względem `1e9` (GB/s), a w części względem `1024\*\*3` (GiB/s).
Skutek: potencjalnie mylące porównania.

5.6 Windows: brak kompletnej ścieżki build dla CPU

Wczytywanie `microbench.dll` jest przewidziane, ale brak analogicznej, utrzymywanej ścieżki budowania jak dla Linux/macOS.
Skutek: "multi-platform" nie jest domknięte dla Windows.

6. Rekomendacje wdrożeniowe (kolejność)

1. Ujednolicić schemat CSV i `benchmark` naming dla wszystkich benchmarków CPU.
2. Naprawić `run\_pointer\_latency.py` (spójny `uint32` end-to-end).

3. Dostosować ``analysis/cpu_summary.py`` do finalnego schematu i dodać walidację brakujących kolumn.
4. Ujednolicić metryki energii na ``EnergyLogger`` (CPU i GPU).
5. Ustalić jedną konwencję jednostek przepustowości (``GB/s`` albo ``GiB/s``) i stosować ją globalnie.
6. Domknąć ścieżkę `build+run` dla Windows (CPU, a docelowo także GPU zależnie od backendu).

7. Checklist końcowy

7.1 Co już jest

- [x] Benchmarki CPU: bandwidth (ST/MT), STREAM (ST/MT), pointer latency, FMA, FMA peak.
- [x] Kernels CPU w C i integracja przez ``ctypes``.
- [x] Benchmarki GPU dla CUDA/HIP/Metal/OpenCL.
- [x] Orkiestracja CPU/GPU (``run_all_*`
- [x] Zapis wyników do CSV z metadanymi sprzętowymi.
- [x] Podstawowe skrypty podsumowujące (``analysis/cpu_summary.py``, ``analysis/gpu_summary.py``).
- [x] Pomiar energii dostępny (best-effort) dla CPU i GPU.

7.2 Co jeszcze trzeba dodać / poprawić

- [] Jeden, spójny kontrakt CSV dla wszystkich benchmarków CPU i GPU.
- [] Ujednolicenie ``benchmark`` naming tak, aby summary obejmowało 100% wyników.
- [] Naprawa typu danych w pointer-chasing (``uint32`` spójnie).
- [] Walidacja danych wejściowych do summary (czytelne błędy przy brakujących kolumnach).
- [] Ujednolicenie metryk energii/mocy i źródła pomiaru (CPU/GPU).
- [] Ujednolicenie jednostek przepustowości i jawne oznaczenie w CSV.
- [] Stabilny pipeline `build/run` dla Windows (CPU minimum, docelowo pełny zakres).
- [] Krótki "reproducibility guide" (pinning rdzeni, governor, temperatura, warmup policy).
- [] Zestaw testów smoke/regression dla benchmark driverów i parserów summary.

8. Proponowane artefakty docelowe po domknięciu

- Jeden standard CSV schema (``docs/CSV_SCHEMA.md``).
- Jeden standard metryk (``docs/METRICS.md``).
- Jedna instrukcja uruchamiania per platforma (``docs/RUN_LINUX.md``, ``docs/RUN_MACOS.md``, ``docs/RUN_WINDOWS.md``).
- Raport porównawczy CPU/GPU generowany automatycznie z aktualnych danych.